Implémentation, optimisation et diffusion dalgorithmes dunranking

Nouriddin Boukhzar & Ayoub Salhi
Encadrant: Amaury Curiel

Sorbonne université

Contents

1 Introduction **3**

2 Tâches à réaliser **4**

2.1 Algorithmes d'unranking 4

2.2 Application web 4

2.3 Contribution à SymPy 4

3 Les algorithmes (ordre combinatoire) **5**

3.1 Partitions de Stirling 5

3.1.1 Formule de comptage 5

3.1.2 Interprétation combinatoire 5

3.1.3 Implémentation 5

3.2 Partitions de Stirling Ordonnées 6

3.2.1 Formule de comptage 6

3.2.2 Interprétation combinatoire 6

3.2.3 Implémentation 7

3.3 Partitions de Lah 8

3.3.1 Formule de comptage 8

3.3.2 Interprétation combinatoire 8

3.3.3 Implémentation 8

3.4 Partitions de Lah Ordonnées 9

3.4.1 Formule de comptage 9

3.4.2 Interprétation combinatoire 10

3.4.3 Implémentation 10

3.5 Partitions Entières 11

3.5.1 Formule de comptage 11

3.5.2 Interprétation combinatoire 11

3.5.3 Implémentation 12

4 Application Web **13**

4.1 Fonctionnement 13

4.2 Justification des choix techniques 13
4.2.1 Gestion de version 13
4.2.2 React + Vite + TypeScript 13
4.2.3 Exécution dans un Web Worker 14
4.3 Présentation 14

Chapter 1

Introduction

Dans les années 1960, le mathématicien Gian-Carlo Rota a identifié 12 problèmes fondamentaux en combinatoire. Si les formules de comptage associées à ces problèmes sont connues depuis longtemps, ce n'est que récemment que des algorithmes permettant de générer chacune de ces structures de manière systématique ont été proposés.

Ce projet a pour objectif d'implémenter ces algorithmes, de les optimiser via la programmation concurrente, puis de les intégrer à la bibliothèque Python SymPy, référence en calcul symbolique. En parallèle, une application web sera développée afin de rendre ces algorithmes accessibles et visualisables en ligne.

Chapter 2

Tâches à réaliser

2.1 Algorithmes d'unranking

La première tâche consiste à implémenter les algorithmes d'unranking pour chacun des 12 problèmes de Rota. Ces algorithmes permettent, à partir d'un entier, de retrouver directement la structure combinatoire associée sans énumérer toutes les structures précédentes.

2.2 Application web

Dans le même temps, une application web sera développée pour permettre d'exécuter ces algorithmes en ligne et d'en visualiser le fonctionnement. L'objectif est de rendre ces outils accessibles sans nécessiter d'installation locale.

2.3 Contribution à SymPy

Enfin, les algorithmes implémentés seront adaptés aux standards de la bibliothèque open-source SymPy, afin d'y être intégrés et mis à disposition de la communauté. Cette étape implique de respecter les conventions de code, de documentation et de tests propres au projet.

Chapter 3

Les algorithmes (ordre combinatoire)

3.1 Partitions de Stirling

3.1.1 Formule de comptage

Le nombre de partitions de Stirling de $\{1, \ldots, n\}$ en k blocs est donné par les nombres de Stirling de seconde espèce, notés $\left\{ {n \atop k} \right\}$, définis par la récurrence suivante :

$$\left\{ {n \atop k} \right\} = \begin{cases} 1 & \text{si } k = 1 \text{ ou } k = n \\ \left\{ {n-1 \atop k-1} \right\} + k \cdot \left\{ {n-1 \atop k} \right\} & \text{sinon} \end{cases}$$

3.1.2 Interprétation combinatoire

Cette récurrence se comprend naturellement : pour construire une k-partition de $\{1, \ldots, n\}$, on considère le placement de l'élément n. Soit on crée un nouveau bloc singleton $\{n\}$, en complétant une $(k-1)$-partition de $\{1, \ldots, n-1\}$, ce qui donne $\left\{ {n-1 \atop k-1} \right\}$ possibilités, soit on insère n dans l'un des k blocs d'une k-partition existante de $\{1, \ldots, n-1\}$, ce qui donne $k \cdot \left\{ {n-1 \atop k} \right\}$ possibilités.

3.1.3 Implémentation

Les nombres de Stirling croissent très rapidement, ce qui nécessite l'utilisation de grands entiers. TypeScript offre un support natif pour cela via le type `bigint`. Un cache LRU permet de mémoïser les appels récursifs et d'éviter les recalculs. Ces détails ont été omis des exemples de code ci-dessous par souci de clarté.

```
function stirling_numbers(n: number, k: number): bigint {
  if (n === 0 && k === 0) return 1n;
  if (n === 0 || k === 0) return 0n;
  const res = stirling_numbers(n - 1, k - 1) + BigInt(k) *
  ↪  stirling_numbers(n - 1, k);
  return res;
}
function unrank_stirling(n: number, k: number, r: number): number[][] {
  if (n === 0 || k === 0) return [];
  const cntA = stirling_numbers(n - 1, k - 1);
  if (BigInt(r) < cntA) {
    const res = unrank_stirling(n - 1, k - 1, r);
    res.push([n]);
    return res;
  }
  const [pos, r2] = divmod(BigInt(r) - cntA, stirling_numbers(n - 1, k));
  const res = unrank_stirling(n - 1, k, Number(r2));
  res[Number(pos)].push(n);
  return res;
}
```

3.2 Partitions de Stirling Ordonnées

3.2.1 Formule de comptage

Le nombre de partitions de Stirling ordonnées de $\{1, \ldots, n\}$ en k blocs est noté O_k^n et satisfait la récurrence suivante :

$$O_k^n = \begin{cases} k! & \text{si } k = 1 \text{ ou } k = n \\ k \cdot (O_{k-1}^{n-1} + O_k^{n-1}) & \text{sinon} \end{cases}$$

3.2.2 Interprétation combinatoire

Une partition de Stirling ordonnée est une partition de Stirling dont les blocs sont ordonnés. La récurrence se comprend comme suit : pour construire une k-partition de $\{1, \ldots, n\}$, soit on ajoute n comme nouveau bloc singleton dans l'une des k positions possibles parmi les blocs existants d'une $(k-1)$-partition de $\{1, \ldots, n-1\}$, soit on insère n dans l'un des k blocs d'une k-partition existante de $\{1, \ldots, n-1\}$. Dans les deux cas, on multiplie par k, d'où la relation $O_k^n = k \cdot (O_{k-1}^{n-1} + O_k^{n-1})$.

Aux cas de base, si $k = 1$ tous les éléments sont dans un seul bloc et il n'y a qu'une seule partition. Si $k = n$ chaque élément est dans son propre bloc, et toute permutation de ces blocs donne une partition différente, soit $k!$ partitions.

3.2.3 Implémentation

L'algorithme suit la même logique que pour les partitions de Stirling classiques, à ceci près que l'ordre des blocs est significatif.

```
function ordered_stirling_numbers(n: number, k: number): bigint {
  if (k === 1 || k === n) return factorial(k);
  const result = BigInt(k) * (ordered_stirling_numbers(n - 1, k - 1) +
  ↪  ordered_stirling_numbers(n - 1, k));
  return result;
}
function unrank_perm(n: number, r: number): number[] {
  const elems = Array.from({ length: n }, (_, i) => i + 1);
  const perm: number[] = [];
  let r2 = BigInt(r);
  for (let m = n; m > 0; m--) {
    const f = factorial(m - 1);
    const [idx, newR] = divmod(r2, f);
    r2 = newR;
    perm.push(elems.splice(Number(idx), 1)[0]);
  }
  return perm;
}
function unrank_ordered_stirling(n: number, k: number, r: number):
↪  number[][] {
  if (k === 1) return [Array.from({ length: n }, (_, i) => i + 1)];
  if (k === n) return unrank_perm(n, r).map(x => [x]);
  // Case A: n is a singleton block inserted at one of k positions
  const cntA = BigInt(k) * ordered_stirling_numbers(n - 1, k - 1);
  if (r < cntA) {
    const [pos, newR] = divmod(BigInt(r), ordered_stirling_numbers(n - 1,
    ↪  k - 1));
    const res = unrank_ordered_stirling(n - 1, k - 1, newR);
    res.splice(Number(pos), 0, [n]);
    return res;
  }
  // Case B: n joins one of the k existing blocks
```

```
  const [pos, newR] = divmod(BigInt(r) - cntA, ordered_stirling_numbers(n - 1, k));
  const res = unrank_ordered_stirling(n - 1, k, newR);
  res[Number(pos)].push(n);
  return res;
}
```

3.3 Partitions de Lah

3.3.1 Formule de comptage

Le nombre de partitions de Lah de $\{1, \ldots, n\}$ en k blocs est donné par les nombres de Lah, notés $\left\lfloor {n \atop k} \right\rfloor$, définis par la récurrence suivante :

$$\left\lfloor {n \atop k} \right\rfloor = \begin{cases} n! & \text{si } k = 1 \\ 1 & \text{si } k = n \\ \left\lfloor {n-1 \atop k-1} \right\rfloor + (n-1+k) \cdot \left\lfloor {n-1 \atop k} \right\rfloor & \text{sinon} \end{cases}$$

3.3.2 Interprétation combinatoire

Une partition de Lah est une partition dont les blocs sont des séquences ordonnées, mais dont l'ensemble des blocs est lui-même non ordonné. Autrement dit, l'ordre des éléments *au sein* de chaque bloc compte, mais l'ordre des blocs entre eux ne compte pas.

La récurrence se comprend comme suit : pour construire une k-partition de $\{1, \ldots, n\}$, soit on ajoute n comme nouveau bloc singleton, en complétant une $(k-1)$-partition de $\{1, \ldots, n-1\}$, soit on insère n dans l'un des k blocs existants d'une k-partition de $\{1, \ldots, n-1\}$. Dans ce dernier cas, n peut être placé en tête ou après n'importe lequel des $n-1$ éléments déjà présents, répartis dans k blocs, ce qui donne $n-1+k$ positions possibles.

Aux cas de base, si $k = 1$ tous les éléments forment une seule séquence, soit $n!$ arrangements possibles. Si $k = n$ chaque élément est seul dans son bloc, et il n'existe qu'une seule telle partition.

3.3.3 Implémentation

```
function lah_numbers(n: number, k: number): bigint {
  if (n === 0 && k === 0) return 1n;
  if (n === 0 || k === 0) return 0n;
  if (n === k) return 1n;
  const res = lah_numbers(n - 1, k - 1) + BigInt(n + k - 1) *
  ↪  lah_numbers(n - 1, k);
  return res;
}
function unrank_lah(n: number, k: number, r: number): number[][] {
  if (n === 0) return [];
  if (n === k) return Array.from({ length: n }, (_, x) => [x + 1]);
  const cntA = lah_numbers(n - 1, k - 1);
  // Case A: n is a new singleton block
  if (r < cntA) return [...unrank_lah(n - 1, k - 1, r), [n]];
  // Case B: insert n into one of the existing blocks
  const [blockIdx, r2] = divmod(BigInt(r) - cntA, lah_numbers(n - 1, k));
  const res = unrank_lah(n - 1, k, r2);
  let pos = Number(blockIdx);
  for (const block of res) {
    if (pos < block.length + 1) {
      block.splice(pos, 0, n);
      break;
    }
    pos -= block.length + 1;
  }
  return res;
}
```

3.4 Partitions de Lah Ordonnées

3.4.1 Formule de comptage

Le nombre de partitions de Lah ordonnées de $\{1, \ldots, n\}$ en k blocs est noté $\left\lceil {n \atop k} \right\rceil$ et satisfait la récurrence suivante :

$$\left\lceil {n \atop k} \right\rceil = \begin{cases} n! & \text{si } k = 1 \text{ ou } k = n \\ k \cdot \left\lceil {n-1 \atop k-1} \right\rceil + (n - 1 + k) \cdot \left\lceil {n-1 \atop k} \right\rceil & \text{sinon} \end{cases}$$

3.4.2 Interprétation combinatoire

Une partition de Lah ordonnée est la structure la plus riche parmi les quatre partitions étudiées : l'ordre des éléments au sein de chaque bloc compte, et l'ordre des blocs entre eux compte également. Elle combine ainsi les caractéristiques des partitions de Stirling ordonnées (blocs ordonnés) et des partitions de Lah (séquences internes).

La récurrence se comprend comme suit : pour construire une k-partition de $\{1, \ldots, n\}$, soit on ajoute n comme nouveau bloc singleton dans l'une des k positions possibles parmi les blocs d'une $(k-1)$-partition de $\{1, \ldots, n-1\}$, soit on insère n dans l'un des blocs existants d'une k-partition de $\{1, \ldots, n-1\}$, en $n-1+k$ positions possibles.

Aux cas de base, si $k = 1$ tous les éléments forment une seule séquence, soit $n!$ arrangements. Si $k = n$ chaque élément est seul dans son bloc et toute permutation des blocs donne une partition différente, soit également $n!$ partitions.

3.4.3 Implémentation

```
function ordered_lah_numbers(n: number, k: number): bigint {
  if (k === 1 || k === n) return factorial(n);
  const result =
    BigInt(k) * ordered_lah_numbers(n - 1, k - 1) +
    BigInt(n - 1 + k) * ordered_lah_numbers(n - 1, k);
  return result;
}
function unrank_ordered_lah(n: number, k: number, r: number): number[][]
↪ {
  if (n === 0) return [];
  if (k === 1) return [unrank_perm(n, r)];
  if (k === n) return unrank_perm(n, r).map(x => [x]);
  // Case A: n is a new singleton block inserted at one of k positions
  const cntA = BigInt(k) * ordered_lah_numbers(n - 1, k - 1);
  if (r < cntA) {
    const [pos, newR] = divmod(BigInt(r), ordered_lah_numbers(n - 1, k -
    ↪ 1));
    const res = unrank_ordered_lah(n - 1, k - 1, newR);
    res.splice(Number(pos), 0, [n]);
    return res;
  }
  // Case B: insert n into one of the existing ordered blocks
```

```
  const [blockIdx, newR] = divmod(BigInt(r) - cntA, ordered_lah_numbers(n - 1, k));
  const res = unrank_ordered_lah(n - 1, k, newR);
  let pos = Number(blockIdx);
  for (const block of res) {
    if (pos < block.length + 1) {
      block.splice(pos, 0, n);
      return res;
    }
    pos -= block.length + 1;
  }
  return res;
}
```

3.5 Partitions Entières

3.5.1 Formule de comptage

Le nombre de partitions de l'entier n en k parts est noté $p(n, k)$ et satisfait la récurrence suivante :

$$p(n,k) = \begin{cases} 0 & \text{si } n < 1 \\ 1 & \text{si } k = 1 \text{ ou } k = n \\ p(n-1, k-1) + p(n-k, k) & \text{sinon} \end{cases}$$

3.5.2 Interprétation combinatoire

Une partition entière de n en k parts est un multiensemble de k entiers strictement positifs dont la somme vaut n. Contrairement aux partitions d'ensembles vues précédemment, il n'y a pas d'éléments à distribuer : on décompose un entier en parts.

La récurrence se comprend comme suit : pour construire une k-partition de n, soit la plus petite part vaut 1, et il reste à partitionner $n - 1$ en $k - 1$ parts, soit toutes les parts sont strictement supérieures à 1, auquel cas on peut soustraire 1 à chacune des k parts, ce qui ramène au problème de partitionner $n - k$ en k parts.

Aux cas de base, si $k = 1$ la seule partition possible est $\{n\}$, et si $k = n$ la seule partition possible est $\{1, 1, \ldots, 1\}$.

3.5.3 Implémentation

```
function int_partitions(n: number, k: number): bigint {
  if (n < 1) return 0n;
  if (k === 1 || n === k) return 1n;
  const result = int_partitions(n - 1, k - 1) + int_partitions(n - k, k);
  return result;
}
function unrank_int_partitions(n: number, k: number, r: number): number[] {
  if (n < 1) return [];
  if (k === 1) return [n];
  if (n === k) return Array(n).fill(1);
  // Case A: smallest part is 1
  const cntA = int_partitions(n - 1, k - 1);
  if (r < cntA) return [1, ...unrank_int_partitions(n - 1, k - 1, r)];
  // Case B: all parts > 1, subtract 1 from each
  return unrank_int_partitions(n - k, k, BigInt(r) - cntA).map(i => i + 1);
}
```

Chapter 4

Application Web

4.1 Fonctionnement

L'application doit permettre d'exécuter les algorithmes d'unranking directement depuis un navigateur, sans installation locale ni serveur de calcul. Cela impose que l'ensemble de la logique algorithmique s'exécute côté client. Par ailleurs, certains calculs peuvent être longs pour de grandes valeurs de n et k, ce qui soulève la question de la réactivité de l'interface.

4.2 Justification des choix techniques

4.2.1 Gestion de version

Le code source est versionné avec Git et disponible à l'adresse suivante: `https://github.com/qyrhx/Unranking-Algos`. L'utilisation de Git facilite le suivi des modifications et la collaboration entre les membres de l'équipe.

4.2.2 React + Vite + TypeScript

L'application est développée avec React, Vite et TypeScript. React permet de structurer l'interface de manière claire et réactive. Vite offre un environnement de développement rapide et une chaîne de compilation moderne. TypeScript apporte la sûreté des types, particulièrement utile pour manipuler des `bigint` et des structures combinatoires potentiellement complexes. La bibliothèque `lru-cache` est également utilisée pour la mémoïsation des fonctions de comptage, dont les valeurs sont réutilisées intensivement lors des appels récursifs.

4.2.3 Exécution dans un Web Worker

Les algorithmes d'unranking sont exécutés dans un *Web Worker*, c'est-à-dire un thread séparé du thread principal du navigateur. Cela présente plusieurs avantages :

- L'interface reste réactive pendant le calcul l'utilisateur peut toujours interagir avec la page.
- Les calculs lourds ne bloquent pas le rendu visuel.

En contrepartie, cette approche implique quelques limitations :

- La communication entre le thread principal et le worker se fait par échange de messages, ce qui introduit une légère latence.
- Le worker ne peut pas accéder directement au DOM.
- En cas de calcul trop long, il n'est pas possible d'interrompre proprement le thread depuis l'extérieur : la seule solution est de le terminer entièrement.

C'est ce dernier point qui a motivé l'ajout d'un bouton **Abort** : lorsqu'un calcul semble bloquer, l'utilisateur peut terminer le worker en cours et en recréer un nouveau immédiatement, ce qui restitue le contrôle sans avoir à recharger la page.

4.3 Présentation

L'interface se compose d'un formulaire permettant de saisir les paramètres n, k et r, de sélectionner l'algorithme souhaité, et de lancer différentes actions.

Le bouton **Unrank** retourne la structure combinatoire correspondant au rang r donné. Le bouton **Count** affiche le nombre total de structures pour les paramètres choisis. Le bouton **Random r** tire un rang aléatoire uniformément dans $\{0, \ldots, \text{count}(n, k) - 1\}$ et affiche la structure correspondante, en mettant à jour le champ r pour permettre la reproduction du résultat.

Le bouton **List all** énumère toutes les structures dans l'ordre lexicographique, limité à 1000 entrées pour éviter de saturer l'interface. Le nombre total est toujours affiché, et un avertissement signale si la liste est tronquée.

Enfin, lorsqu'un calcul est en cours, un bouton **Abort** apparaît, permettant d'interrompre le worker et d'en recréer un nouveau sans recharger la page.

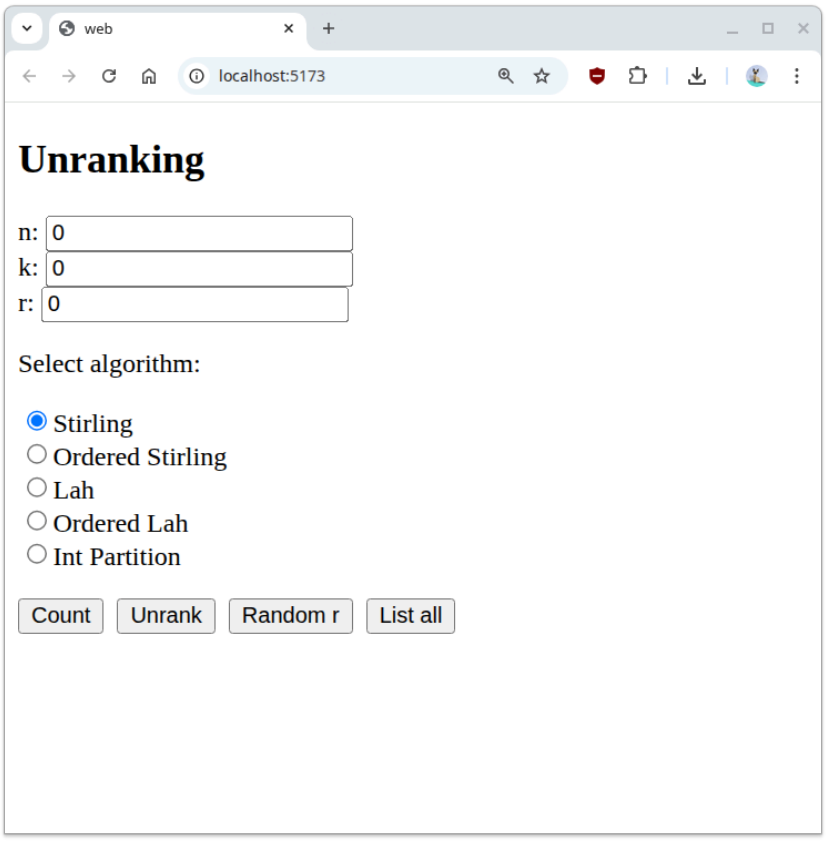

Figure 4.1: Interface principale de l'application

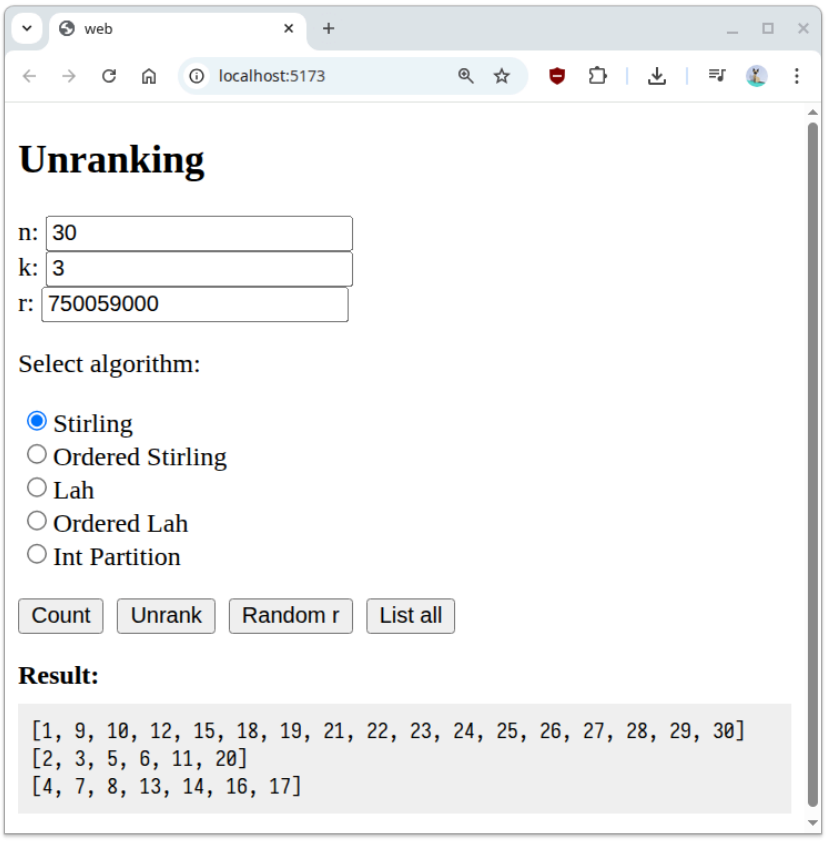

Figure 4.2: Résultat d'un unranking pour les partitions de Stirling

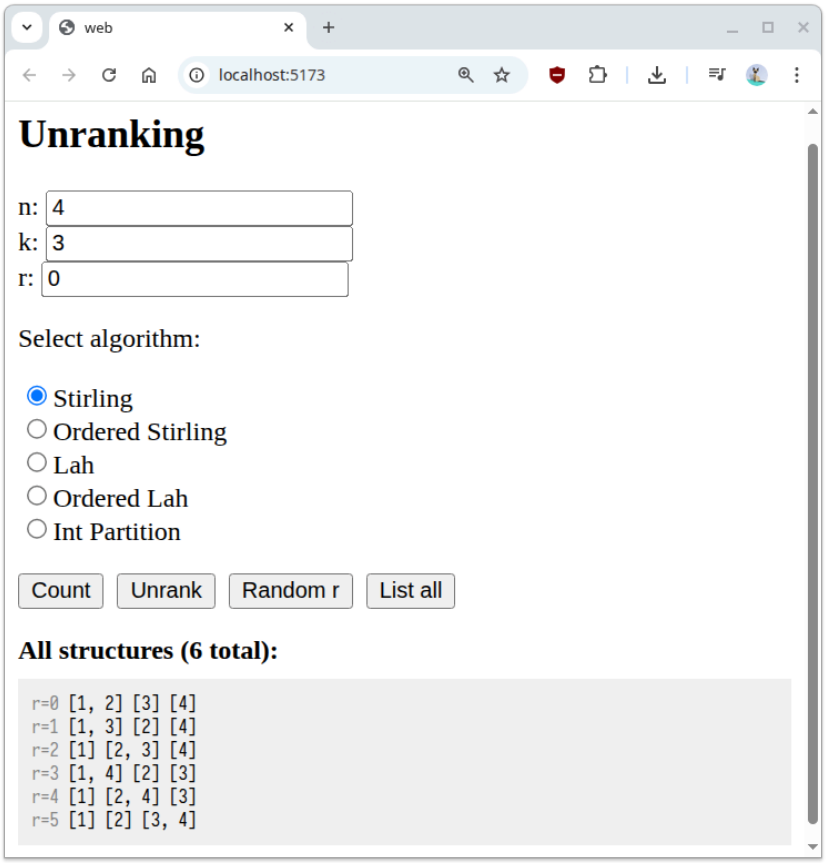

Figure 4.3: Liste des structures pour de petites valeurs de n et k

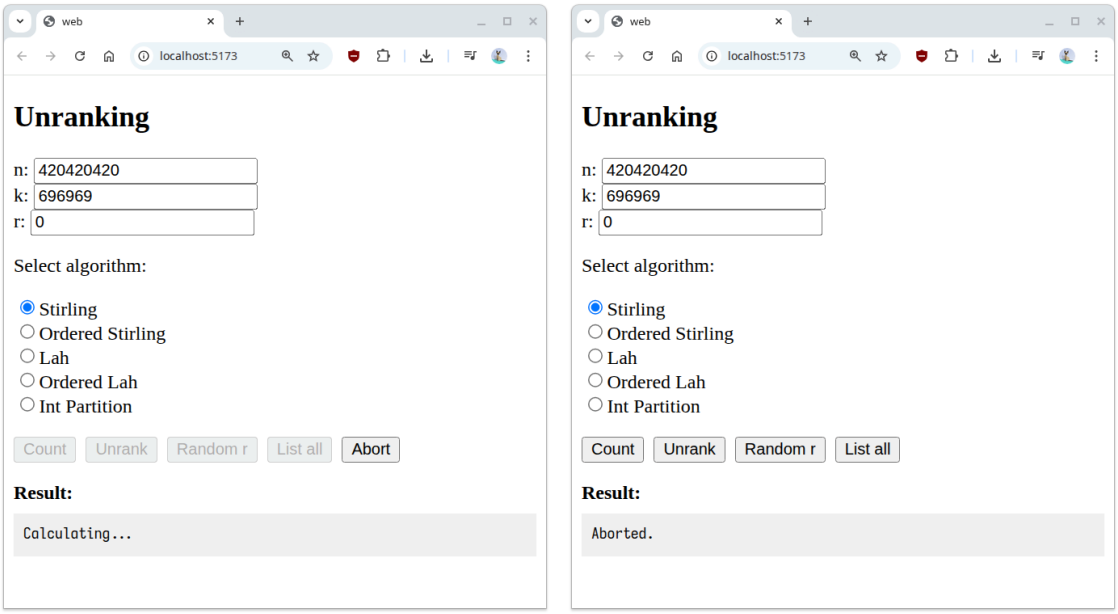

Figure 4.4: Bouton Abort visible pendant un calcul long